Palm OS Programming Recipes

Recipe: Using static fields to display non editable text

Fields are normally thought of as being for user input, but they are also used to automatically word-wrap noneditable text. Labels are better if you have just one line of text which never changes, but fields are used when the given text needs to be word-wrapped or scrolled, or if it needs to allow the user to select some text and copy it.

Create a field for display of static text as follows:

1. Create a field on your form; size it as necessary. Set its object identifier ("First" in this example).
2. It should be Usable, not Editable, not Underlined, not Single Line, not Dynamic Size, and Left Justified.
3. Set "Max Characters" to a sufficiently large amount, or else call FldSetMaxChars() at runtime when you know the actual length of your string.
4. Your font should usually be either "Standard" or "Bold". Bold is commonly used to indicate uneditable text, but the Standard font should generally be used when you are displaying large amounts of text.
5. Turning on "Has Scroll Bar" does not mean a scrollbar will automatically appear and operate; you have to write code to do that. For this example, leave this attribute off.

When the form is opening, you need to give the field a pointer or handle to the text to display. How you handle frmOpenEvent will depend on where your text is coming from: typically either from a resource in your application, or a buffer to which you have a pointer. (You could also construct a field by calling FldInsert() one or more times, and use the field's ability to allocate and manage memory for the string, but this is usually less desirable because it takes more memory and time.)

If you have a string resource in your PRC which contains the text to display, do the following in your frmOpenEvent handler, before calling FrmDrawForm():

```
FieldType *fldP;
FormType* frmP;
MemHandle h;

frmP = FrmGetActiveForm();
h = DmGetResource(strRsc, SpeechString);
fldP = (FieldType *) FrmGetObjectPtr(frmP,
                            FrmGetObjectIndex(frmP, MainFirstField));
FldSetMaxChars(fldP, MemHandleSize(h));
FldSetTextHandle(fldP, h);
```

But watch out - when the form is closing, if the field is still using this string handle, it will try to free it - and that would be bad since that handle is to a resource in your application. (Actually, the OS detects that the handle is in the storage heap and will give a Fatal Alert.) So your frmCloseEvent handler will look like this:

```
case frmCloseEvent: {

   frmP = FrmGetActiveForm();
```

```
      fldP = (FieldType *) FrmGetObjectPtr(frmP,
                              FrmGetObjectIndex(frmP, MainFirstField));

      // Tell the field to forget about the handle it is displaying,
      // so that it won't try to free it, since it is in our PRC.
      FldSetTextHandle(fldP, 0);
      break;
   }
```

Why does this give a zero handle to the field? When you give a handle to a field, it "owns" it, and it is in charge of managing that memory, including resizing the handle of an editable field. So if you don't want the field to free a handle, you must still tell it to use another handle (or give it a zero handle). But if the handle is just in the dynamic heap and you want it to be freed, do nothing and it'll happen automatically.

If instead of a string resource, you create or have been given a pointer to a buffer containing the text to display, and if you're sure that buffer won't move or become invalid until the form is closed (or until you give it another pointer or handle), then use code like this to have the field use that buffer directly:

```
   char *strBuffer;

   strBuffer = (char *) MemPtrNew(80);
   StrCopy(strBuffer, "This is a test of allocating a buffer.");

   fldP = (FieldType *) FrmGetObjectPtr(frmP,
                              FrmGetObjectIndex(frmP, MainSecondField));
   FldFreeMemory(fldP);  // initialize everything, just in case.
   FldSetMaxChars(fldP, StrLen(strBuffer));
   FldSetTextPtr(fldP, strBuffer);
   FldRecalculateField(fldP, true);
```

Of course that buffer must be freed when the form is closing. When given a pointer instead of a handle, the Field Manager won't try to free the pointer for you, so you must do it yourself:

```
   case frmCloseEvent: {

      frmP = FrmGetActiveForm();

      // Free the buffer we allocated for this field and gave
      // it as a pointer. This is only OK since we know the field
      // is about to be destroyed, and because we know it is
      // a pointer-based field.
      fldP = (FieldType *) FrmGetObjectPtr(frmP,
                              FrmGetObjectIndex(frmP, MainSecondField));
      MemPtrFree((MemPtr) FldGetTextPtr(fldP));
      break;
   }
```

If on the other hand, to allocate strBuffer in the above example you used MemHandleNew instead of MemPtrNew, and you didn't need the handle for anything else, it would be fine to not do anything in your frmCloseEvent and have the field dispose of the handle itself.